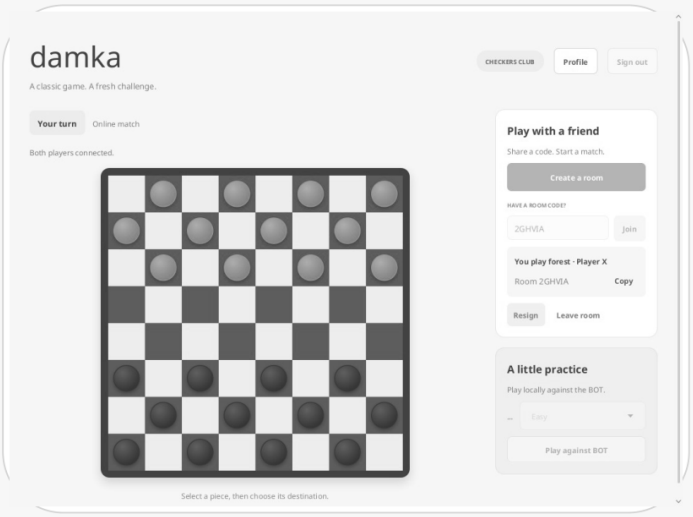
Damka

Checkers, connected.

TCP multiplayer · Local BOT
Accounts, statistics and history



Muthanna Farhat
Rabea Abu Aad
Supervisor: Inbal Ronen

Slide 1

One game, two ways to play

Keep both screens in sync — and remember the result.

Online matches

Two computers
One room code

Local BOT

Three difficulty levels
Offline practice

Personal profiles

Statistics
Match history

One shared rules engine: Game

Slide 2

Built with Java — from interface to storage

Java 17

Object-oriented game and server code

JavaFX + CSS

Desktop screens, styling and animation

TCP / java.net

Socket / ServerSocket · UTF-8

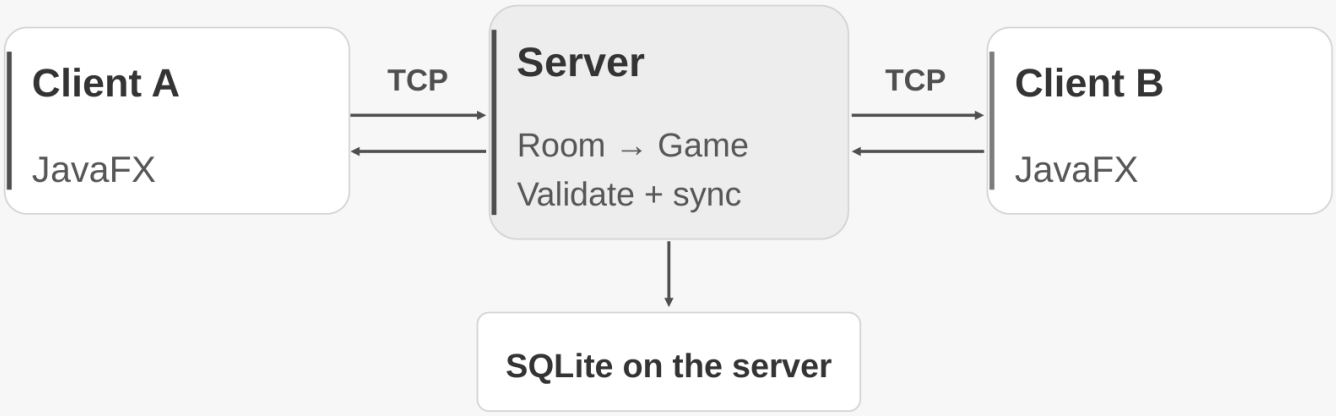
SQLite + JDBC

Accounts, results, statistics and history

Build: Gradle 8.8 **Tests: JUnit 5** **Version control: Git + GitHub**

Slide 3

Why a client–server design?

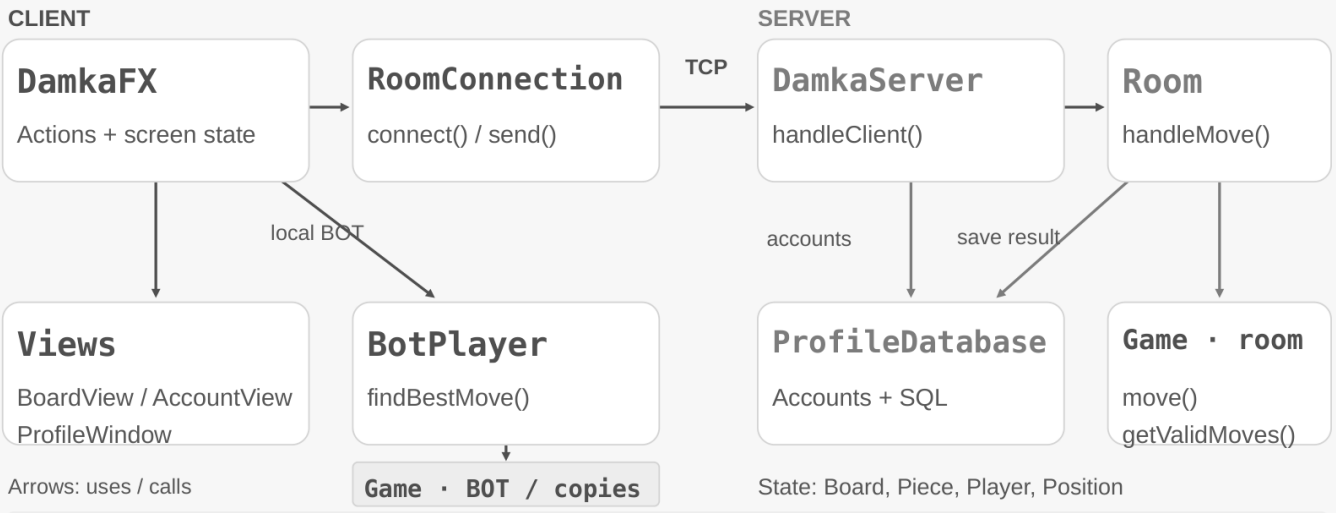


Choice: persistent TCP for two-way updates

Alternative: HTTP polling

Slide 4

Main classes and their responsibilities



Game rules are reused; rooms, local BOT and search use separate game instances.

Slide 5

From a click to both screens

Client A

Select a piece and destination

```
MOVE 5 0 4 1
```

Server

Room: identity + turn

Game: check move rules

```
game.move(from, to)
```

Both clients

Accepted move

```
1  MOVE_APPLIED
2  BOARD
3  TURN
```

Animate → display the accepted state

UTF-8 · one message per line · network work off the UI thread

Slide 6

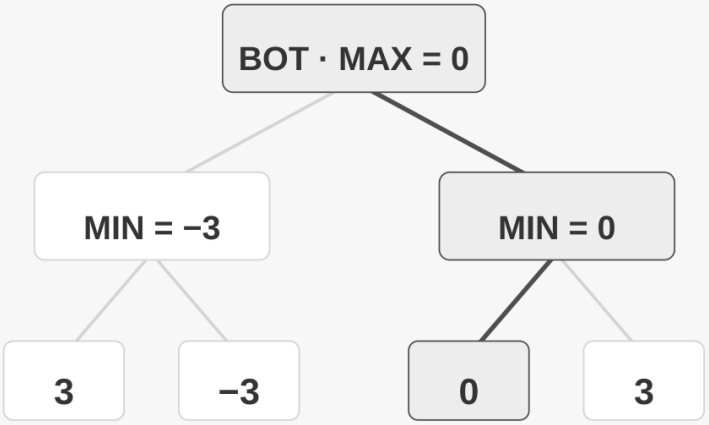
Minimax: how the BOT chooses

Shared game rules

Legal moves on copies

Piece = 3 King = 6

Depth: 1 / 2 / 3



Illustrative two-step search

Alpha–Beta skips branches that cannot improve the decision.

Slide 7

Accounts that remember results

Player profile

Illustrative account

ONLINE	BOT
1 win / 1 match	0 matches

Match history

Online	WIN	Resignation

`profiles`

Username · salt · hash

`game_results`

Players · result · mode · time

Saved results → statistics

Slide 8

How we checked the system

48 automated tests

Game rules

Invalid moves, turns and capture chains

TCP behavior

Two real connections
Matching board updates

Stored results

Authentication
Persistence + deduplication

Manual UI checks: two windows, error messages and movement animation

Automated count: documented passing run, 12 Oct 2026

Slide 9

Engineering challenges and our solutions

Online consistency

Challenge
Two screens, one game

Solution
Server validates moves
Both clients update

Network + UI timing

Challenge
Updates during animation

Solution
Background network I/O
Queued UI updates

Rules + BOT search

Challenge
Consistent move rules

Solution
One Game rules engine
Copies for BOT search

Learning curve: JavaFX after HTML · practical sockets after network theory

Slide 10

Our contribution — with AI assistance

Our decisions

Server owns online state
Shared Game rules
Authenticated result saving

Our validation

Run two clients
Identify UI + animation issues
Review and trace the flow

AI assistance

Generate + revise code
Explain behavior
Help with tests + documents

Responsibility areas
Rabea Abu Aad — Game Logic
Muthanna Farhat — TCP Communication

Slide 11

What we built and learned

One authoritative game.
Two synchronized clients.
Persistent player results.

Next: chat

Inside the room
Over existing TCP

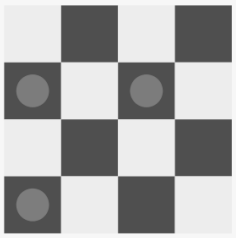
Learning: JavaFX, practical networking,
and integration across UI, rules and storage.

github.com/MuthannaFrhat/damka-game

Slide 12

THANK YOU

Questions & discussion



github.com/MuthannaFrhat/damka-game

Slide 13